Streaming Data Engineering — Complete Handbook

event time, windows, watermarks, state, exactly-once, Spark Structured Streaming, Flink, Kafka Streams, CDC, architectures, operations and interview scenarios

notes by Sagar · Jan 2026

Contents

1. How to use this handbook
2. What streaming is and when to use it
3. Building blocks of a streaming system
4. Time in streaming
5. Windows
6. Watermarks and late data
7. State and stateful processing
8. Delivery guarantees and exactly-once
9. Joins in streaming
10. Spark Structured Streaming: the model
11. Spark Structured Streaming: building pipelines
12. Spark Structured Streaming: operations
13. Other engines: Apache Flink
14. Other engines: Kafka Streams, ksqlDB and cloud services
15. Streaming transport: Kafka-style logs in brief
16. Change Data Capture (CDC)
17. Streaming architectures
18. Common streaming pipeline patterns
19. Data quality in streaming
20. Monitoring and alerting
21. Performance tuning
22. Sizing a streaming pipeline
23. Testing streaming pipelines
24. Troubleshooting
25. Remember it: memory hooks for streaming
26. Interview questions and answers
27. Streaming coding questions with answers
28. Glossary

Sagar

1. How to use this handbook

Streaming feels hard because a few new ideas appear at once: data never ends, it arrives late and out of order, and the system must remember things between events. Once those ideas click, every streaming tool looks similar.

Stage	Read	Goal
Foundations	Chapters 2–6	Understand what streaming is, time, windows and watermarks
State and correctness	Chapters 7–9	State, delivery guarantees, exactly-once, joins
Hands-on	Chapters 10–12	Spark Structured Streaming end to end, with code
Wider view	Chapters 13–16	Flink, Kafka Streams, cloud services, CDC
Design	Chapters 17–19	Architectures, pipeline patterns, data quality
Production	Chapters 20–24	Monitoring, tuning, sizing, testing, troubleshooting
Revision	Chapters 25–28	Memory hooks, interview Q&A, coding questions, glossary

The three ideas that unlock streaming: (1) the data never ends, so you compute on **windows** of it; (2) events arrive **late and out of order**, so you decide how long to wait (**watermark**); (3) the system must **remember** things between events (**state**) and survive crashes without losing or double-counting (**checkpoints + idempotent sinks**).

2. What streaming is and when to use it

Batch vs streaming in simple words

> **In simple words:** *batch is like collecting all the day's letters and sorting them at night. Streaming is like a post office that sorts each letter the moment it arrives. Batch is simpler; streaming is faster.*

	Batch	Streaming
Data	Bounded: a file, a day, a table	Unbounded: never ends
When it runs	On a schedule (hourly, daily)	Continuously, or in very frequent small batches
Latency	Minutes to hours	Milliseconds to minutes
Complexity	Lower	Higher: late data, state, ordering, 24×7 operations
Cost	Compute only while running	Usually always-on compute
Reprocessing	Rerun the job for a date	Replay from the source or from stored history

Latency vocabulary

Term	Typical latency	Example
Real-time (hard)	Microseconds–milliseconds	Engine control, trading systems (not data engineering)
Real-time / low latency	< 1 second	Fraud scoring, ad bidding
Near real-time	Seconds to a few minutes	Live dashboards, alerting, CDC replication
Micro-batch	Seconds to minutes per batch	Spark Structured Streaming default
Frequent batch	15 min – 1 hour	Incremental batch jobs

When streaming is worth it

- The **value of the data drops quickly**: fraud, outages, stock-outs, security threats, live operations.
- **Continuous sources** where batching only adds delay: clickstream, IoT, logs, CDC.
- **Event-driven products**: notifications, recommendations updated per action.

When it isn't

- Reports read once a day — an hourly or daily batch is simpler and cheaper.
- Heavy joins across many large historical tables — batch or incremental batch fits better.
- The team can't operate 24×7 systems yet.

Ask the business **"what decision changes if the data is 1 minute old instead of 1 hour old?"** If nothing changes, you probably need incremental batch, not streaming.

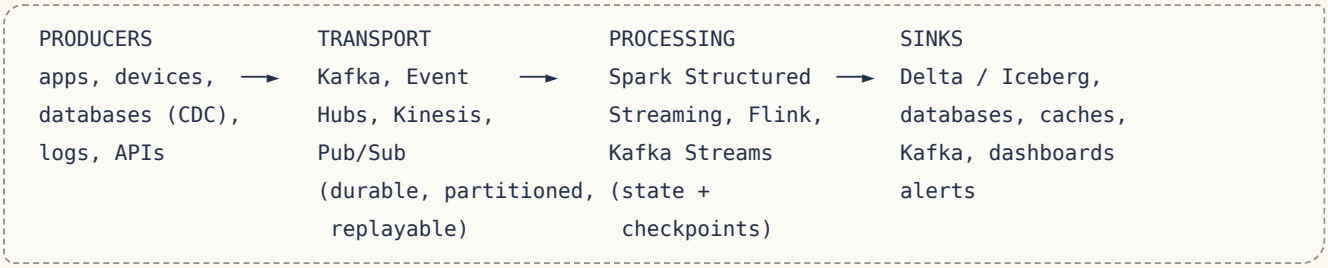
The middle ground: incremental batch

Many "streaming" pipelines are really **incremental batch**: run every 15–60 minutes, process only new data, stop. Spark's **`trigger(availableNow=True)`** and Databricks Auto Loader make this easy: the same streaming code, run on a schedule, with checkpoints tracking what's done. It gives most of the freshness at a fraction of the operational cost.

Sagar

3. Building blocks of a streaming system

The pieces



Events

An event is a fact about something that happened: "order 1001 placed at 10:00:01 for ₹2,500". Good events are:

- **Immutable** — you don't change past events; you add new ones.
- **Self-describing** — include an ID, an event timestamp, the entity key and a schema version.
- **Keyed** — the key (order ID, user ID, device ID) decides ordering and partitioning.

```
{"event_id": "e-8f2c", "event_type": "order_placed", "event_ts": "2026-01-15T10:00:01Z",
 "order_id": 1001, "customer_id": 42, "amount": 2500.00, "schema_version": 3}
```

The log (transport)

A durable, partitioned, append-only log (Kafka and similar) sits between producers and processors:

- **Decouples** producers and consumers — they can fail, scale and deploy independently.
- **Buffers** spikes — if processing slows, events wait in the log instead of being lost.
- **Replays** — consumers can re-read history after a bug fix.
- **Orders** events per partition (by key).

Streams and tables (the duality)

> **In simple words:** a **stream** is the bank statement (every transaction); a **table** is the current balance. You can always compute the balance from the statement, and every change to the balance is a new line on the statement.

- Aggregating a stream by key gives a table (latest value / running total per key).
- The changes to a table form a stream (a changelog — exactly what CDC produces).

This duality explains CDC, log compaction, materialized views and stateful processing.

4. Time in streaming

Three kinds of time

Time	Meaning	Reliable for analytics?
Event time	When the event actually happened (inside the data)	Yes — the truth you usually want
Ingestion time	When the event entered the log/broker	Sometimes
Processing time	When the stream processor handles it	No — depends on delays, restarts, backlogs

> **In simple words:** event time is when you **wrote** the letter; processing time is when the post office **sorted** it. If a letter sits in a mailbox for two days, sorting by arrival would put it on the wrong day.

Why events arrive late and out of order

- Mobile apps offline (flights, tunnels) send events hours later.
- Network retries and partitions.
- Gateways buffering IoT data during outages.
- Multiple sources with different delays.
- A stuck or restarted processor working through a backlog.

Event-time skew

The gap between event time and processing time is always moving: small in normal operation, huge after an outage. A correct streaming job must produce the same answer whether events arrive on time or after a 3-hour backlog — that is what event-time processing plus watermarks gives you.

> **Memory hook:** "Count by when it happened, not when it arrived."

Sagar

5. Windows

You can't "count all clicks" on a never-ending stream and finish. You count clicks ***per window*** *— per 5 minutes, per hour, per user session.*

> ***In simple words:*** *a window is a bucket of time. Each event drops into the bucket(s) its timestamp belongs to; when a bucket is complete, you report its result.*

Window types

```
TUMBLING (fixed, no overlap)          |--10:00--|--10:05--|--10:10--|
  every event in exactly one window

SLIDING / HOPPING (fixed, overlapping) |----10:00-10:10----|
  size 10 min, slide 5 min                    |----10:05-10:15----|
  an event can be in several windows                |----10:10-10:20----|

SESSION (gap-based, per key)          [e e e e]  ...30 min gap...  [e e]
  window closes after inactivity       session 1                   session 2

GLOBAL (one window, custom trigger)    everything, emitted on a custom condition
```

Window	Use for	Spark
Tumbling	Counts per minute/hour, periodic KPIs	`F.window("ts", "5 minutes")`
Sliding	Moving averages, "last 10 minutes, updated every minute"	`F.window("ts", "10 minutes", "1 minute")`
Session	User sessions, device activity bursts	`F.session_window("ts", "30 minutes")`
Global / custom	Count-based or rule-based triggers	Arbitrary stateful processing

> ***Memory hook — "Tiles, Sliding doors, Sessions":*** ***tumbling*** *windows sit side by side like floor* ***tiles****;* ***sliding*** *windows overlap like* ***sliding doors****;* ***session*** *windows stretch like a phone call — they end when nobody talks for a while.*

Window boundaries

- *Windows are aligned to the epoch by default (10:00, 10:05...), not to the first event.*
- *A window* ***[10:00, 10:05)*** *includes 10:00:00 and excludes 10:05:00.*
- *Sliding windows multiply state and output: a 1-hour window sliding every minute keeps 60 windows per key.*

6. Watermarks and late data

The problem

If events can arrive late, when is a window "done"? Wait forever and you never produce results (and state grows without limit). Don't wait at all and late events are wrong or lost.

The watermark

> **In simple words:** a watermark is the "last entry" rule at a restaurant. "We'll accept guests up to 15 minutes late. After that, we close the bill." It's the system's estimate that **no more events older than this time will arrive**.

In Spark: **watermark = max event time seen so far − allowed lateness.**

```
counts = (events
  .withWatermark("event_ts", "15 minutes")          # allowed lateness
  .groupBy(F.window("event_ts", "5 minutes"), "page")
  .count())
```

```
max event_ts seen = 10:32   allowed lateness = 15 min   → watermark = 10:17
window [10:10, 10:15) ends before 10:17 → finalized, emitted (append mode), state dropped
event at 10:12 arriving now          → older than watermark → dropped (too late)
event at 10:20 arriving now          → accepted, its window still open
```

What watermarks control

1. **When results are final** (append mode emits a window once the watermark passes its end).
2. **When state can be deleted** (keeps memory bounded).
3. **Which late events are dropped.**

Choosing the lateness

Smaller watermark	Larger watermark
Lower latency for final results	More complete results
Less state, less memory	More state, more memory
More late events dropped	Results emitted later

Measure it: compute the distribution of **processing_time − event_time** from real data and pick a lateness that covers, say, 99–99.9% of events.

Handling events later than the watermark

- **Accept the loss** for live dashboards (approximate is fine).
- **Batch correction**: land all raw events in a table, and recompute final numbers in a daily batch from the complete data — the "speed layer + correction" pattern.
- **Side output / late table**: some engines (Flink) route late events to a side output; in Spark, compare event time with a watermark yourself in **foreachBatch** and write late

events separately.

Multiple watermarks

When a query has several inputs (e.g. stream-stream join), Spark by default uses the **minimum** of the input watermarks (safe: waits for the slowest stream). It can be set to max (**spark.sql.streaming.multipleWatermarkPolicy=max**) to favour latency, at the risk of dropping data from the slower stream.

> **Memory hook:** *"The watermark is the bouncer: late guests beyond the limit don't get in, and once the bouncer moves on, the old tables are cleared."*

Sagar

7. State and stateful processing

Stateless vs stateful

Stateless (each event alone)	Stateful (remembers across events)
filter, map, select, parse, enrich with a static table	aggregations, windows, deduplication, joins between streams, sessions, pattern detection
Easy to scale and recover	Needs a state store, checkpoints, cleanup (watermarks/timeouts)

Where state lives

- **In memory with snapshots** (Spark's default HDFS-backed state store): fast, but state must fit in executor memory.
- **RocksDB** (embedded key-value store on local disk): handles much larger state with less GC pressure. Default choice for large state in Spark and Flink.
- State is checkpointed to durable storage so it survives failures.

```
spark.conf.set("spark.sql.streaming.stateStore.providerClass",
               "org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider")
```

Keeping state bounded

State grows forever unless something removes it:

- **Watermarks** for windowed aggregations, dedup and joins.
- **Timeouts** for custom stateful logic (processing-time or event-time timeouts).
- **TTL** settings in newer APIs and in Flink.

> A stateful query without a watermark or timeout is a memory leak with a delay. Every stateful operator needs a rule for when state can be forgotten.

Arbitrary stateful processing

When built-in aggregations aren't enough (custom session logic, fraud velocity, alert cool-downs), write your own state logic per key:

- **PySpark:** **applyInPandasWithState** (Spark 3.4+); Spark 4.0 adds the more flexible **transformWithState** API with multiple state variables, timers and TTL.
- **Scala/Java:** **mapGroupsWithState** / **flatMapGroupsWithState**.
- **Flink:** **KeyedProcessFunction** with value/list/map state and timers.

```
import pandas as pd
from pyspark.sql.streaming.state import GroupStateTimeout

def velocity(key, pdfs, state):
    count = state.get[0] if state.exists else 0
    for pdf in pdfs:
        count += len(pdf)
    state.update((count,))
    state.setTimeoutDuration(60 * 60 * 1000)          # forget a card after 1h of inactivity
    if state.hasTimedOut:
        state.remove(); return
    yield pd.DataFrame({"card_id": [key[0]], "txn_count": [count]})

out = (txns.groupBy("card_id")
       .applyInPandasWithState(velocity, outputStructType="card_id string, txn_count long",
                               stateStructType="count long", outputMode="update",
                               timeoutConf=GroupStateTimeout.ProcessingTimeTimeout))
```

*In real code, check **state.hasTimedOut** first — on a timeout call there are no new rows, and you remove the state and optionally emit a final result.*

Sagar

8. Delivery guarantees and exactly-once

The three guarantees

Guarantee	Meaning	Typical cause
At-most-once	Never duplicated, can be lost	Commit position before processing
At-least-once	Never lost, can be duplicated	Commit after processing; retries after failures — the common default
Exactly-once (effectively-once)	Each event affects the result once	Replayable source + checkpoints + idempotent/transactional sink

In simple words: at-least-once is a courier who might deliver the same parcel twice after a mix-up; exactly-once means the receiver checks the parcel ID and ignores the duplicate. "Exactly-once" is really **effectively-once**: duplicates may happen internally, but the final result counts each event once.

Where duplicates come from

1. **Producer retries** — a write succeeded, the ack was lost, the producer sends again.
2. **Consumer restarts** — processed but crashed before committing the position; reprocessed after restart.
3. **Upstream systems** — the source itself sends the same event twice (mobile SDK retries).
4. **Reprocessing / backfills** — replaying data into a sink that already has it.

The recipe: R·C·I

Memory hook — R·C·I, "Really Careful Ingestion": **R**eplayable source + **C**heckpointed progress + **I**dempotent (or transactional) sink. Miss any one and you get gaps or duplicates.

Part	Spark / Kafka example
Replayable source	Kafka offsets, files in storage, Delta table versions
Checkpoint	Spark checkpoint: offsets + state + commit log per micro-batch
Idempotent sink	Delta (atomic commit per batch, batch IDs tracked), MERGE by key, upsert by primary key

Idempotent sinks in practice

Sink	How to make writes idempotent
Delta Lake	Built in for append (Spark records batch IDs); **foreachBatch** + MERGE by key for upserts; **txnAppId** + **txnVersion** options for idempotent writes inside **foreachBatch**
Relational database	Upsert by primary key (**MERGE**, **INSERT ... ON CONFLICT**); or store last batch ID per target
Key-value store / cache	**SET key = value** is naturally idempotent
Kafka (as sink)	Idempotent producer + transactions for Kafka-to-Kafka; otherwise consumers dedup by event ID
Files	Deterministic file names per batch; atomic rename/commit
External APIs	Idempotency keys (event ID) if the API supports them

Deduplicating source duplicates

Even with exactly-once processing, if the **source** sends duplicates you must deduplicate on a business/event ID:

```
deduped = (events.withWatermark("event_ts", "30 minutes")
                 .dropDuplicatesWithinWatermark(["event_id"]))      # Spark 3.5+
```

Or MERGE into the target by **event_id**, or use **ROW_NUMBER()** per key in silver.

9. Joins in streaming

Stream–static join

```
enriched = clicks.join(F.broadcast(dim_products), "product_id", "left")
```

- The static side is re-read each micro-batch (Delta tables pick up new versions), so dimension updates flow in.
- Cheap and common: enrichment with product, customer, device metadata.
- Inner and left outer (stream on the left) are supported.

Stream–stream join

Joins two unbounded streams — for example ad impressions with clicks, orders with payments.

```
imp = impressions.withWatermark("imp_ts", "1 hour")
clk = clicks.withWatermark("click_ts", "2 hours")

joined = imp.join(clk, F.expr("""
    click_ad_id = imp_ad_id AND
    click_ts BETWEEN imp_ts AND imp_ts + INTERVAL 1 HOUR"""), "leftOuter")
```

- Both sides are buffered in state until a match is impossible.
- **Watermarks on both sides + a time-range condition** are required so state can be cleaned. Without them, state grows forever (and outer joins need them to emit unmatched rows).
- Left outer rows with no match are emitted only after the watermark passes — so they appear with a delay.

Joining with changing reference data

Situation	Approach
Small dimension, changes occasionally	Stream–static join on a Delta table
Dimension changes often and history matters	Point-in-time lookup: SCD2 table joined on event time in **foreachBatch**
Very large dimension	Key-value lookup (cache/DB) inside **mapInPandas**/**foreachBatch**, batched
Dimension arrives as a stream (CDC)	Stream–stream join, or maintain it as a table (compacted topic / Delta) and use stream–static

10. Spark Structured Streaming: the model

In simple words: *Structured Streaming runs your normal DataFrame code again and again on small batches of new data, remembering where it stopped in a* **checkpoint**. *You write it almost like batch code.*

The unbounded table

Each trigger, new rows are appended to an imaginary **input table**. *Your query defines a* **result table**. *The* **output mode** *decides what is written to the sink each trigger.*

```
trigger 1: input rows 1–100   → result table v1 → write (append/update/complete)
trigger 2: input rows 101–180 → result table v2 → write changes
...        Spark computes only the incremental work, using state for aggregations
```

Micro-batch execution

1. Ask the source for new data since the last offsets (e.g. Kafka offsets, new files).
2. Write the planned offsets to the checkpoint's **offset log** (write-ahead).
3. Run the query on that batch as a normal Spark job (with state lookups/updates).
4. Write results to the sink.
5. Record the batch as done in the **commit log**.

If it crashes between 2 and 5, the restart re-runs the same batch with the same offsets — and idempotent sinks make that safe.

Triggers

Trigger	Behaviour	Use for
Default (none)	Next batch starts as soon as the previous finishes	Lowest micro-batch latency
`processingTime="1 minute"`	One batch per interval	Steady, predictable load
`availableNow=True`	Process everything available now (in multiple batches), then stop	Scheduled incremental batch jobs
`once=True`	One batch then stop (older; prefer availableNow)	Legacy
`continuous="1 second"`	Experimental low-latency mode, limited operations (no aggregations)	Rarely used

Output modes

Mode	Writes	Works with
append	Only new final rows; aggregation rows only after the watermark passes the window	Stateless queries; windowed aggregations with watermark; stream-stream joins
update	Rows changed in this batch	Aggregations where the sink supports upserts (foreachBatch + MERGE)
complete	The entire result table every batch	Small aggregations only (state never dropped)

Memory hook — "Add, Update, Copy everything": append = only add finished rows; **update** = only changed rows; **complete** = rewrite the whole table every time.

Sources and sinks

Sources	Sinks
Kafka / Event Hubs (Kafka API) / Kinesis (connector)	Delta / Parquet / files
Files in a directory (CSV, JSON, Parquet...); Auto Loader on Databricks	Kafka
Delta tables (as a stream of appends; change data feed for updates)	**foreachBatch** — any batch writer (JDBC, MERGE, multiple sinks)
Rate / rate-micro-batch (testing)	**foreach** (row-by-row, rarely needed)
Socket (testing only)	Console / memory (testing)

II. Spark Structured Streaming: building pipelines

A complete Kafka → bronze → silver → gold pipeline

```
from pyspark.sql import functions as F, Window as W

# BRONZE: land raw events with Kafka metadata
raw = (spark.readStream.format("kafka")
       .option("kafka.bootstrap.servers", BROKERS)
       .option("subscribe", "orders.events.v1")
       .option("startingOffsets", "earliest")          # used only when there's no checkpoint
       .option("maxOffsetsPerTrigger", 500_000)
       .load())

(raw.select(F.col("key").cast("string"), F.col("value").cast("string"),
            "topic", "partition", "offset", F.col("timestamp").alias("kafka_ts"))
    .withColumn("_ingest_ts", F.current_timestamp())
    .writeStream.format("delta")
    .option("checkpointLocation", "/chk/orders_bronze")
    .trigger(processingTime="1 minute")
    .toTable("bronze.order_events"))

# SILVER: parse, validate, dedup
schema = "event_id STRING, order_id LONG, customer_id LONG, status STRING, amount
DECIMAL(18,2), event_ts TIMESTAMP"
parsed = (spark.readStream.table("bronze.order_events")
          .select(F.from_json("value", schema).alias("e"), "partition", "offset")
          .select("e.*", "partition", "offset"))

good = parsed.filter("event_id IS NOT NULL AND event_ts IS NOT NULL")
bad  = parsed.filter("event_id IS NULL OR event_ts IS NULL")

(bad.writeStream.format("delta").option("checkpointLocation", "/chk/orders_quarantine")
    .toTable("quarantine.order_events"))

(good.withWatermark("event_ts", "30 minutes")
     .dropDuplicatesWithinWatermark(["event_id"])
     .writeStream.format("delta").option("checkpointLocation", "/chk/orders_silver")
     .trigger(processingTime="1 minute").toTable("silver.order_events"))

# GOLD: 5-minute revenue per status (append after the watermark)
(spark.readStream.table("silver.order_events")
   .withWatermark("event_ts", "10 minutes")
   .groupBy(F.window("event_ts", "5 minutes"), "status")
   .agg(F.count("*").alias("orders"), F.sum("amount").alias("revenue"))
   .writeStream.format("delta").outputMode("append")
   .option("checkpointLocation", "/chk/orders_gold_5m")
   .toTable("gold.orders_5m"))
```

Current state with foreachBatch + MERGE

```
from delta.tables import DeltaTable

def upsert_orders(batch_df, batch_id):
    latest = (batch_df
      .withColumn("rn", F.row_number().over(
          W.partitionBy("order_id").orderBy(F.desc("event_ts"), F.desc("offset"))))
      .filter("rn = 1").drop("rn"))
    (DeltaTable.forName(batch_df.sparkSession, "silver.orders_current").alias("t")
       .merge(latest.alias("s"), "t.order_id = s.order_id")
       .whenMatchedUpdateAll(condition="s.event_ts >= t.event_ts")   # ignore older events
       .whenNotMatchedInsertAll()
       .execute())

(spark.readStream.table("silver.order_events")
   .writeStream.foreachBatch(upsert_orders)
   .option("checkpointLocation", "/chk/orders_current")
   .trigger(processingTime="2 minutes").start())
```

Why this pattern matters:

- Deduplicate **within** the micro-batch (several events per key) before MERGE — MERGE fails if multiple source rows match one target row.
- The **event_ts >= t.event_ts** condition protects against out-of-order events overwriting newer state.
- If a batch is retried, the MERGE produces the same result — idempotent.

Writing one stream to several sinks

```
def fan_out(df, batch_id):
    df.persist()
    df.write.format("delta").mode("append").saveAsTable("silver.events")
    (df.filter("severity = 'HIGH'")
       .select(F.col("event_id").alias("key"), F.to_json(F.struct("*")).alias("value"))
       .write.format("kafka").option("kafka.bootstrap.servers", BROKERS)
       .option("topic", "alerts.v1").save())
    df.unpersist()

stream.writeStream.foreachBatch(fan_out).option("checkpointLocation", "/chk/fanout").start()
```

foreachBatch itself is at-least-once for external sinks — on a retry the same **batch_id** is processed again. Make each write idempotent, or record processed **batch_id**s in the target.

Sagar

File sources and Auto Loader

```
# open-source Spark file source
(spark.readStream.schema(schema).option("maxFilesPerTrigger", 500)
      .json("abfss://raw@lake.dfs.core.windows.net/events/"))

# Databricks Auto Loader: scalable discovery, schema inference and evolution
(spark.readStream.format("cloudFiles")
      .option("cloudFiles.format", "json")
      .option("cloudFiles.schemaLocation", "/meta/schemas/events")
      .load("abfss://raw@lake.dfs.core.windows.net/events/"))
```

Reading a Delta table as a stream

```
spark.readStream.table("silver.orders")                          # new appends only
spark.readStream.option("readChangeFeed", "true").option("startingVersion", 120) \
     .table("silver.orders")                                     # inserts, updates, deletes (CDF)
spark.readStream.option("skipChangeCommits", "true").table("t")  # ignore updates/deletes in source
```

A plain Delta stream fails if the source table has updates/deletes (they're not appends) — use change data feed or **skipChangeCommits** deliberately.

Sagar

12. Spark Structured Streaming: operations

Checkpoints

The checkpoint folder holds:

Folder	Content
offsets/	Planned source offsets per batch (write-ahead log)
commits/	Batches that completed successfully
state/	State store files per operator and partition
sources/	Source-specific metadata (e.g. file source seen files)
metadata	Query ID

Rules:

- **One checkpoint per query.** Never share, never copy between queries.
- Don't delete it to "fix" a problem — the query restarts from **startingOffsets** (reprocessing or skipping data).
- Store it on durable storage (cloud storage, HDFS), not local disk.
- Some code changes are **not compatible** with an existing checkpoint.

Which query changes are safe on restart?

Change	Safe with the same checkpoint?
Filters, projections, adding columns to output (stateless)	Generally yes
Changing trigger interval, **maxOffsetsPerTrigger**	Yes
Adding/removing a Kafka topic to **subscribe**	Generally yes (new topic starts per **startingOffsets**)
Changing grouping keys, aggregations, window sizes, watermark column	**No** — state schema changes
Changing stream-stream join logic or dedup columns	**No**
Changing number of shuffle partitions for a stateful query	**No** — state is partitioned by it (it's fixed in the checkpoint)
Changing the sink type	Usually not

For incompatible changes: start a **new checkpoint** and decide where to start — reprocess from bronze/earliest, or from a known offset/version — and make the sink idempotent so overlap is harmless.

Sagar

Graceful stop and restart

```
q = df.writeStream ... .start()
q.status          # is a batch running? waiting for data?
q.lastProgress    # input rows, rates, durations, offsets, state metrics
q.stop()          # stops after interrupting the current batch; safe with checkpoint + idempotent
sink
```

For a clean stop at a batch boundary, wait until **`q.status["isTriggerActive"]`** is false, then call **`stop()`**. In production, an external marker file or job control can request the stop.

Reading the progress metrics

```
"numInputRows"            rows in this batch
"inputRowsPerSecond"      arrival rate
"processedRowsPerSecond"  processing rate         → must stay ≥ input rate over time
"durationMs.triggerExecution"   batch duration    → should be < trigger interval
"stateOperators[].numRowsTotal" / "memoryUsedBytes"  state size → must not grow forever
"sources[].startOffset / endOffset / latestOffset"   how far behind the source you are
```

StreamingQueryListener (monitoring hook)

```
from pyspark.sql.streaming import StreamingQueryListener

class Monitor(StreamingQueryListener):
    def onQueryStarted(self, e): log.info("started %s", e.name)
    def onQueryProgress(self, e):
        p = e.progress
        push_metrics(p.name, p.inputRowsPerSecond, p.processedRowsPerSecond, p.batchDuration)
    def onQueryIdle(self, e): pass
    def onQueryTerminated(self, e):
        if e.exception: alert(f"stream {e.id} failed: {e.exception}")

spark.streams.addListener(Monitor())
```

Running streams in production

- Run each stream as its own job (or a few related queries per job) with **automatic restarts** (Databricks continuous jobs / retries, Kubernetes restart policy, YARN AM retries).
- Use **separate scheduler pools** or clusters for heavy streams so one doesn't starve another.
- Keep **bronze append-only** so any downstream stream can be rebuilt by replaying bronze.
- Compact small files regularly (OPTIMIZE, auto-compaction).

13. Other engines: Apache Flink

What it is

Apache Flink is a distributed engine built for **true event-at-a-time streaming** (not micro-batch), with strong event-time and state support. It's widely used for low-latency, complex stateful applications.

Core concepts

Concept	Meaning
JobManager	Coordinates the job, checkpoints, recovery
TaskManager	Workers running parallel task slots
DataStream API	Low-level streaming API (Java/Scala; Python via PyFlink)
Table API / Flink SQL	Declarative SQL over streams and tables
Keyed state	Per-key value/list/map state, with TTL
Timers	Event-time or processing-time callbacks per key
Checkpoints	Periodic distributed snapshots (Chandy-Lamport style barriers) for exactly-once state
Savepoints	Manually triggered snapshots for upgrades, migrations, rescaling
Watermarks	Generated at sources with a strategy (e.g. bounded out-of-orderness)
Side outputs	Extra output streams, e.g. for late events

Flink SQL example

```
CREATE TABLE clicks (
  user_id STRING, page STRING, ts TIMESTAMP(3),
  WATERMARK FOR ts AS ts - INTERVAL '10' SECOND
) WITH ('connector' = 'kafka', 'topic' = 'clicks', 'format' = 'json',
        'properties.bootstrap.servers' = 'broker:9092', 'scan.startup.mode' = 'latest-offset');

SELECT window_start, window_end, page, COUNT(*) AS views
FROM TABLE(TUMBLE(TABLE clicks, DESCRIPTOR(ts), INTERVAL '1' MINUTE))
GROUP BY window_start, window_end, page;
```

Spark Structured Streaming vs Flink

	Spark Structured Streaming	Apache Flink
Model	Micro-batch (continuous mode experimental)	Event-at-a-time
Typical latency	~100 ms to minutes	Milliseconds to seconds
State & event time	Good; watermarks, RocksDB, arbitrary state APIs	Very rich: timers, TTL, side outputs, savepoints
Batch + streaming	Same DataFrame API, strong batch ecosystem	Unified too, but batch ecosystem smaller
Lakehouse integration	Excellent (Delta, Databricks)	Good (Iceberg, Paimon, Delta connector)
Python	First-class (PySpark)	PyFlink exists; Java/Scala most mature
Best for	Streaming ETL into the lakehouse, near-real-time analytics	Low-latency, complex event-driven apps, CEP

Memory hook: *"Spark streams in buses (micro-batches leave every few seconds); Flink streams in taxis (each event goes immediately)."*

Sagar

14. Other engines: Kafka Streams, ksqlDB and cloud services

Kafka Streams

A Java library (not a cluster) for Kafka-to-Kafka processing inside your application.

- **KStream** (event stream) and **KTable** (changelog/table), **GlobalKTable** (fully replicated lookup table).
- Local state stores (RocksDB) backed by changelog topics in Kafka — state is restored from Kafka after failures.
- Exactly-once with **processing.guarantee=exactly_once_v2** (Kafka transactions).
- Scales by running more instances; parallelism limited by partitions.
- Great for microservices and Kafka-centric event processing; not for writing to a lakehouse.

ksqlDB

SQL over Kafka topics (streams and tables, persistent queries, materialized views) built on Kafka Streams. Good for simple filtering, joins and aggregations without writing Java.

Managed cloud services

Cloud	Transport	Processing
Azure	Event Hubs (Kafka-compatible), IoT Hub	Azure Stream Analytics (SQL), Databricks, Fabric Real-Time Intelligence (Eventstream, KQL)
AWS	Kinesis Data Streams, MSK (Kafka), Firehose (delivery)	Managed Service for Apache Flink, Glue streaming, EMR, Databricks
GCP	Pub/Sub, Managed Kafka	Dataflow (Apache Beam), Dataproc, Databricks

Apache Beam / Dataflow

Beam is a programming model (Java/Python/Go) with one API for batch and streaming, run on runners like Google Dataflow or Flink. It popularized the key streaming questions: **what** is computed, **where** in event time (windows), **when** in processing time results are emitted (triggers), and **how** refinements relate (accumulation).

> **Memory hook — Beam's four questions: "What, Where, When, How": What** results (transforms), **Where** in event time (windows), **When** to emit (watermarks + triggers), **How** to refine (accumulate or discard).

15. Streaming transport: Kafka-style logs in brief

(Detailed in the Kafka handbook; the essentials for any streaming design.)

Concept	Why it matters for streaming
Topic partitions	Unit of parallelism and ordering
Key → partition	Same key, same partition → per-key order
Offsets	Consumer position; the basis for replay and exactly-once
Consumer groups	Parallel consumption; max parallelism = partitions
Retention	How far back you can replay; must exceed worst-case downtime
Compaction	Keeps latest value per key — current-state topics, CDC
Schema Registry	Enforces compatible schema evolution between producers and consumers
DLQ topics	Park bad records without blocking the partition

Partitioning decisions

- Key by the entity whose events must stay ordered (order_id, user_id, device_id).
- Avoid hot keys (one giant customer, a default "unknown" key) — they create one overloaded partition and one slow task.
- Plan the partition count for peak throughput and future growth; changing it later breaks key → partition mapping.

Kinesis and Pub/Sub differences

	Kafka	Kinesis Data Streams	Pub/Sub
Unit of parallelism	Partition	Shard	Managed (no partitions to manage)
Ordering	Per partition	Per shard	Per ordering key (opt-in)
Replay	By offset/time within retention	By sequence number/time within retention	Seek to timestamp/snapshot (with retention enabled)
Delivery to consumer	Pull	Pull (or enhanced fan-out push)	Push or pull, ack-based

16. Change Data Capture (CDC)

What CDC is

> **In simple words:** *CDC is a stream of "what changed" in a database — every insert, update and delete — read from the database's transaction log, in commit order.*

CDC approaches

Approach	How	Pros	Cons
Log-based (Debezium, native CDC, cloud DMS)	Read the transaction log (binlog, WAL, redo, SQL Server CDC)	Captures every change incl. deletes; low source load; ordered	Setup and permissions; log retention to manage
Query-based (timestamp polling)	**`WHERE updated_at > last_watermark`**	Simple	Misses deletes and intermediate changes; needs reliable timestamps; source load
Trigger-based	DB triggers write to audit tables	Captures everything	Adds load and complexity to the source
Snapshot diff	Compare full snapshots	Works anywhere	Expensive, coarse, slow

A CDC event

```
{"op": "u",
 "before": {"id": 42, "city": "Bangalore", "tier": "silver"},
 "after":  {"id": 42, "city": "Bengaluru", "tier": "gold"},
 "ts_ms": 1768471200123,
 "source": {"db": "crm", "table": "customers", "lsn": "0000a1f2:00012ab3:0004"}}
```

`op`: **`c`** create, **`u`** update, **`d`** delete, **`r`** snapshot read.

Applying CDC correctly

1. **Order by the source position** (LSN / binlog position / SCN), not by arrival time.
2. **Deduplicate per key per batch** — keep only the latest change per key for current-state tables.
3. **Guard against old changes** — only apply if the incoming position is newer than the stored one.
4. **Handle deletes** — hard delete in current-state tables; close the version in SCD2 history.
5. **Keep the raw changelog** in bronze for audit and rebuilds.

```
def apply_cdc(batch, batch_id):
    latest = (batch.withColumn("rn", F.row_number().over(
                W.partitionBy("id").orderBy(F.desc("lsn")))).filter("rn = 1"))
    (DeltaTable.forName(batch.sparkSession, "silver.customers").alias("t")
       .merge(latest.alias("s"), "t.id = s.id")
       .whenMatchedDelete(condition="s.op = 'd' AND s.lsn > t.lsn")
       .whenMatchedUpdateAll(condition="s.op <> 'd' AND s.lsn > t.lsn")
       .whenNotMatchedInsertAll(condition="s.op <> 'd'")
       .execute())
```

On Databricks, declarative pipelines offer **AUTO CDC / APPLY CHANGES** with **SEQUENCE BY** and **STORED AS SCD TYPE 1/2**, which implements these rules for you.

Initial snapshot + streaming changes

CDC tools first take a **consistent snapshot** of existing rows, then stream changes from the log position at snapshot time. Downstream must treat snapshot rows (**op = r**) as inserts/upserts and then apply the stream — with ordering by position preventing the snapshot from overwriting newer changes.

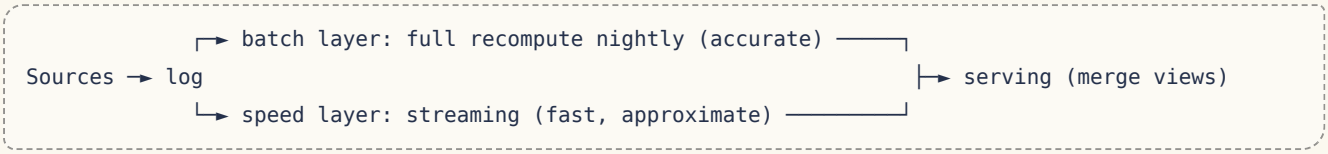
Outbox pattern

Services write business changes and an "outbox" event row in the **same DB transaction**; CDC publishes outbox rows to Kafka. This avoids the dual-write problem (DB updated but event not sent, or vice versa).

Sagar

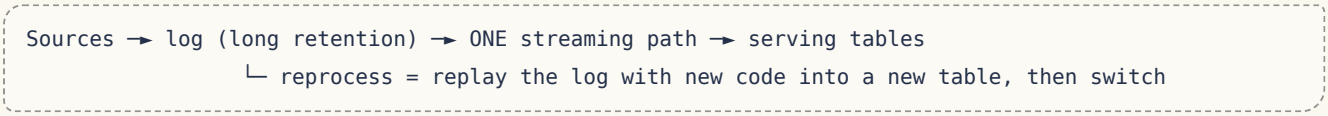
17. Streaming architectures

Lambda architecture



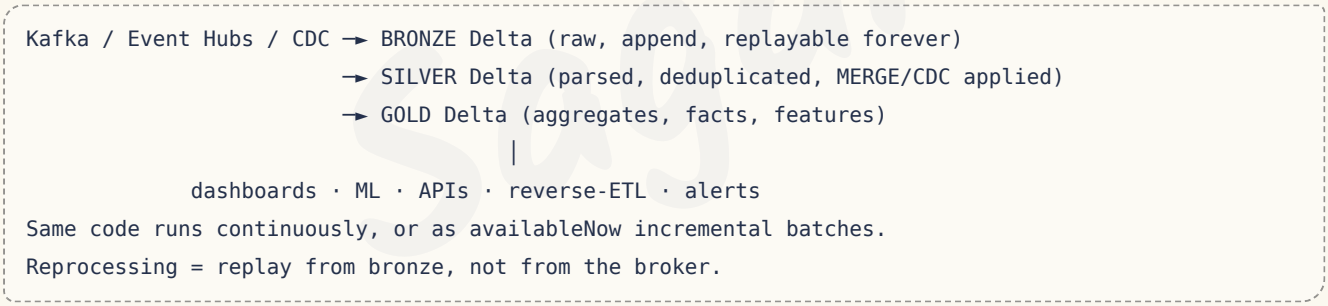
- *Pros: accurate final results, fast recent results.*
- *Cons: **two code paths** for the same logic, hard to keep consistent.*

Kappa architecture



- *Pros: one code base.*
- *Cons: replay of long history can be slow and needs long retention (or a lakehouse copy).*

Streaming lakehouse (most common today)



Memory hook: *"Lambda has two kitchens (fast and slow) cooking the same dish; Kappa has one kitchen that can re-cook from the recipe log; the lakehouse keeps every ingredient in the pantry (bronze) so it can always re-cook."*

Choosing latency per layer

Not everything needs to be real time. A common design: bronze and silver streaming (minutes), gold aggregates as frequent batch, daily corrections from the complete data.

18. Common streaming pipeline patterns

Pattern	What it does	Key techniques
Ingest to lake	Land events from Kafka/files to bronze	Append, checkpoints, keep offsets/metadata
Clean & conform	Parse, validate, dedup, standardize	Explicit schemas, quarantine/DLQ, dedup with watermark
Real-time aggregation	KPIs per window	Tumbling/sliding windows, watermark, append mode
Sessionization	Group events into sessions	Session windows or custom state with timeouts
Enrichment	Add reference data	Stream–static joins, broadcast, periodic refresh
CDC replication	Keep a current-state table in sync	Order by LSN, MERGE, deletes, SCD2 history
Alerting / anomaly	Detect conditions and notify	Windows, thresholds, rate-of-change, cool-down state
Velocity features	Counts/sums per key over recent time	Stateful per-key processing with timeouts
Stream-stream correlation	Match related events (impression → click)	Watermarks on both sides, time-bound join
Fan-out	One stream to many sinks	**foreachBatch**, idempotent writes per sink
Late-data correction	Make final numbers complete	Live approximate + batch recompute from silver
Dead letters	Keep bad events without blocking	DLQ topic / quarantine table + alerts + replay tool

Alert with cool-down (sketch)

```
def raise_alerts(batch, batch_id):
    candidates = batch.filter("avg_value > threshold")
    recent = spark.table("gold.alerts").filter("alert_ts > current_timestamp() - INTERVAL 30 MINUTES")
    new = candidates.join(recent.select("device_id", "rule"), ["device_id", "rule"], "left_anti")
    new.withColumn("alert_ts", F.current_timestamp()).write.mode("append").saveAsTable("gold.alerts")
```

19. Data quality in streaming

Where checks run

Stage	Checks
Producer / schema registry	Schema compatibility, required fields
Bronze → silver	Parse success, required fields not null, valid types/ranges, duplicates
Silver → gold	Business rules, referential checks against dimensions
Continuous monitors	Volume per minute vs baseline, freshness (event-time lag), null rates, DLQ rate

Handling bad records

- **Never block a partition on one bad event** — route it to a DLQ/quarantine with the error reason, and continue.
- Count quarantined records per batch and alert when the rate exceeds a threshold.
- Keep a replay path: fix the bug, then reprocess quarantined events.

Freshness and completeness metrics

- **Event-time lag:** `now − max(event_ts)` in the target — how stale the data is.
- **Source lag:** latest source offset − processed offset.
- **Volume:** events per minute vs the same minute last week; sudden drops often mean a producer bug.

20. Monitoring and alerting

The golden signals for a stream

Signal	Healthy	Alert when
Source lag (records / time)	Stable, small	Growing for N minutes
Input vs processing rate	Processing ≥ input on average	Processing consistently below input
Batch duration vs trigger	Duration < trigger interval	Duration > trigger for several batches
State size	Stable (bounded)	Growing without limit
Event-time freshness	Within SLA	Data older than SLA
Query status	Active	Terminated / restarting repeatedly
DLQ / quarantine rate	Near zero	Spike
Output volume	Within baseline	Sudden drop or spike

Tools

- Spark: **lastProgress**, **StreamingQueryListener**, Spark UI "Structured Streaming" tab, Databricks streaming metrics.
- Kafka: consumer lag exporters; managed-service consoles (Event Hubs, MSK, Confluent).
- Dashboards in Grafana / Datadog / Azure Monitor; alerts to PagerDuty/Teams/Slack.

> Monitor **lag in time**, not only in records — "10 minutes behind" means something to the business; "2 million records behind" doesn't.

21. Performance tuning

Throughput

- Enough **source partitions** (Kafka partitions) for parallelism at peak.
- **minPartitions** (Kafka source) to split large Kafka partitions into more Spark tasks.
- Right-size **shuffle partitions** for stateful queries — fixed in the checkpoint, so choose before first run (e.g. 2–3× total cores).
- **Rate limits**: **maxOffsetsPerTrigger** / **maxFilesPerTrigger** / **maxBytesPerTrigger** to keep batch sizes stable and recover gracefully from backlogs.
- Avoid Python UDFs in hot paths; use built-in functions or pandas UDFs.

Latency

- Shorter triggers (or default trigger) and smaller batches.
- Fewer stateful operators and shuffles; broadcast small dimensions.
- RocksDB state store for large state (less GC); **changelog checkpointing** in newer RocksDB providers reduces checkpoint time.
- Avoid writing huge numbers of small files each batch — balance trigger interval with file size.

State

- Always bound state (watermark/timeout).
- Store aggregates, not raw lists of events.
- Watch **numRowsTotal** and memory per state operator.

Sinks

- Delta: optimized writes / auto-compaction; scheduled OPTIMIZE; liquid clustering on query keys.
- MERGE in **foreachBatch**: keep target clustered on the merge key; include partition/date predicates in the condition to limit files scanned.
- Databases: batch writes, upserts, connection pooling; don't write row by row.

Backlog recovery

After an outage, a stream may face hours of backlog. With rate limits it catches up steadily instead of attempting one giant batch. Scale up temporarily (autoscaling or more workers), then scale back.

Sagar

22. Sizing a streaming pipeline

Start from throughput

```
peak_rate (MB/s)  = events_per_second_at_peak × avg_event_size
kafka_partitions  ≈ peak_rate / per_partition_throughput × headroom (1.5–2)
                    (per-partition throughput ≈ 5–10 MB/s — measure it)
cores             ≈ peak_events_per_sec / events_per_sec_per_core (measured in a test)
```

Worked example

Item	Value
Average rate	20,000 events/s; peak 3× = 60,000 events/s
Event size	1 KB → peak ~60 MB/s
Kafka partitions	60 MB/s ÷ ~5 MB/s ≈ 12 → with headroom **24–48**
Processing per core (parse + dedup + window, measured)	~5,000 events/s
Cores at peak	60,000 ÷ 5,000 = 12 → with headroom **~20–24 cores**
Cluster	3–4 workers of 8 cores (autoscale 2–5)
State	~5M active keys × ~200 bytes ≈ 1 GB (+ overhead) → RocksDB, comfortable
Shuffle partitions (stateful)	~2–3× cores → **48–64**, fixed in checkpoint

Volume per day → average rate

Daily volume	Average	Typical peak (≈3×)
10 GB	~0.12 MB/s	~0.35 MB/s
100 GB	~1.2 MB/s	~3.5 MB/s
1 TB	~11.6 MB/s	~35 MB/s
5 TB	~58 MB/s	~175 MB/s

Always validate with a load test at peak rate plus a **backlog test** (stop the stream for an hour, restart, and measure how long it takes to catch up).

23. Testing streaming pipelines

Unit test the transformation as batch

Write transformations as functions on DataFrames; test them with batch DataFrames — the same function works on a stream.

```
def clean_events(df):
    return (df.filter("event_id IS NOT NULL")
              .withColumn("amount", F.col("amount").cast("decimal(18,2)")))

def test_clean_events(spark):
    df = spark.createDataFrame([("e1", "10.5"), (None, "3")], "event_id string, amount string")
    assert clean_events(df).count() == 1
```

Test streaming behaviour

- **Memory source / rate source** and **trigger(availableNow=True)** to run a stream to completion in tests.
- Write input files in steps, run the stream, assert output after each step — test late data, duplicates, and restarts with the same checkpoint.
- Test **restart idempotency**: run, kill midway, restart with the same checkpoint, check no duplicates or gaps.

```
q = (spark.readStream.schema(schema).json(input_dir)
       .transform(clean_events)
       .writeStream.format("delta").option("checkpointLocation", chk)
       .trigger(availableNow=True).start(out_dir))
q.awaitTermination()
assert spark.read.format("delta").load(out_dir).count() == expected
```

24. Troubleshooting

Symptom	Likely cause	Fix
Lag keeps growing	Processing slower than input; skewed partition; slow sink	Scale cores/partitions, fix hot keys, speed up sink, remove UDFs
Batch duration > trigger interval	Too much data per batch, heavy MERGE, state too big	Rate limits, optimize MERGE (clustering, predicates), RocksDB
State grows forever	Missing watermark/timeout; watermark column not in grouping	Add watermark/timeout; include event-time window/column correctly
Aggregates never appear in append mode	No watermark, or watermark not advancing (no new data)	Add watermark; understand that results emit only after the watermark passes
Duplicates in target	At-least-once retries, source duplicates, non-idempotent foreachBatch	Dedup on event ID, MERGE by key, idempotent writes
Data missing after restart	Checkpoint deleted/changed; retention expired; **failOnDataLoss=false** hid it	Keep checkpoints stable, longer retention, bronze copy for replay
Query fails after a code change	Incompatible change with the checkpoint (state schema)	New checkpoint + controlled reprocessing
Many tiny files	Short triggers, many partitions	Longer triggers, optimized writes, compaction
MERGE error: multiple source rows matched	Several events per key in one batch	Keep latest per key in the batch before MERGE
Stream-stream join uses huge memory	Missing watermarks or time-range condition	Add both, and tighten the time bound
Out-of-order updates overwrite newer data	Applying by arrival, not by event/sequence	Condition MERGE on newer event time / LSN

25. Remember it: memory hooks for streaming

The story: the 24-hour post office

1. **The letter box (producer → log).** Letters (events) are dropped into numbered slots by postcode (partition by key). The slots keep letters for 7 days (retention), and anyone can re-read them (replay).
2. **The date stamp vs the sorting time.** Each letter has the date it was written (event time). The sorter notes when it was sorted (processing time). Reports always go by the written date.
3. **Sorting trays (windows).** Letters go into trays by hour written (tumbling), into overlapping trays for "last 3 hours" (sliding), or into a tray per person that closes when they stop writing for a while (session).
4. **The closing bell (watermark).** "We accept letters up to 1 day late." Once the bell rings for a day, that day's tray is sealed and reported; later letters are set aside.
5. **The ledger (state).** For each customer, the clerk keeps a running total. Old pages are torn out when nobody needs them (state cleanup).
6. **The logbook (checkpoint).** "Last letter processed: slot 7, number 4,512." After a power cut, work resumes from #4,513.
7. **The receipt check (idempotent sink).** Every parcel has an ID; the receiver ignores IDs already received. That's how duplicates don't double-count (exactly-once).
8. **The returns shelf (DLQ).** Unreadable letters go to a shelf with a note, so the line keeps moving.

One-look anchors

Concept	Picture	Hook
Stream vs table	Bank statement vs balance	Statement → balance; balance changes → statement
Event time	Date written on the letter	Count by when it happened
Tumbling window	Floor tiles	Side by side, no overlap
Sliding window	Sliding doors	Overlapping views
Session window	Phone call	Ends after silence
Watermark	Closing bell / bouncer	How late is too late
State	The clerk's ledger	Remember, then forget on time
Checkpoint	The logbook	Resume exactly where you stopped
Exactly-once	Receipt check	R·C·I
DLQ	Returns shelf	Park bad events, keep moving
Micro-batch vs event-at-a-time	Bus vs taxi	Spark vs Flink

Mnemonics

Mnemonic	Meaning
R·**C**·**I** — *Really Careful Ingestion*	Replayable source + Checkpoint + Idempotent sink = exactly-once
Tiles, Sliding doors, Sessions	Tumbling, sliding, session windows
Add, Update, Copy everything	Append, update, complete output modes
What, Where, When, How	Beam's four questions: results, event-time windows, triggers, refinements
Count by when it happened, not when it arrived	Use event time
Every state needs a way to forget	Watermarks, timeouts, TTL
Lag in minutes, not in messages	Monitor time lag for the business
L·**O**·**D**·**S** *of late data*	**L**imit with watermark, **O**utput late to a side table, **D**aily batch correction, **S**ize lateness from real delay data

Sagar

26. Interview questions and answers

Question	Answer
Batch vs streaming?	Bounded vs unbounded data; scheduled vs continuous; minutes–hours vs seconds latency; streaming adds state, late data and 24×7 operations
Event time vs processing time?	When it happened vs when it's processed; use event time for correct analytics
What is a watermark?	Max event time seen minus allowed lateness; decides when windows are final, when state is dropped, and which late events are discarded
Tumbling vs sliding vs session windows?	Fixed non-overlapping; fixed overlapping; gap-based per key
How do you handle late data?	Watermark sized from measured delays; late events to a side table; batch recompute for final numbers
Exactly-once in Spark?	Replayable source + checkpointed offsets/state + idempotent sink (Delta / MERGE); dedup source duplicates on event ID
Append vs update vs complete?	Only final new rows; changed rows; whole result each time
What's in a checkpoint?	Offsets (WAL), commits, state, source metadata
Which code changes break a checkpoint?	Changes to stateful logic: grouping keys, aggregations, windows, joins, dedup keys, shuffle partitions
Stream–static vs stream–stream joins?	Static side re-read each batch, cheap; stream–stream buffers both sides and needs watermarks + time bounds
What is foreachBatch for?	Running batch writers per micro-batch: MERGE upserts, JDBC, multiple sinks — must be idempotent
How do you size Kafka partitions?	Peak throughput ÷ per-partition throughput, plus headroom; also ≥ desired consumer parallelism
Spark vs Flink?	Micro-batch vs event-at-a-time; Spark for lakehouse ETL and near real time; Flink for low latency and complex state
Lambda vs Kappa?	Two paths (batch + speed) vs one streaming path with replay; lakehouse with bronze replay is today's common middle ground
How do you apply CDC correctly?	Order by source LSN, latest per key per batch, apply only newer positions, handle deletes, keep raw changelog
How do you monitor a stream?	Lag (time), input vs processing rate, batch duration, state size, freshness, DLQ rate, query status
Stream is falling behind — what do you check?	Spark UI for skew/slow stages, sink duration, batch size, UDFs, partitions; then scale, rate-limit, fix hot keys
availableNow vs continuous streaming?	availableNow processes the backlog then stops — scheduled incremental batch with streaming guarantees, cheaper than 24×7

Scenario questions

Scenario	Strong answer
"Dashboard counts jump after a stream restart."	Duplicates from reprocessed batches — make the sink idempotent (Delta append / MERGE by event ID), dedup source duplicates with a watermark
"We changed the window size and the stream won't start."	Stateful change incompatible with the checkpoint — new checkpoint, reprocess from bronze for the needed period, idempotent sink
"Mobile events arrive up to 2 days late."	Live windows with a short watermark for the dashboard; all events land in silver; daily batch recompute for final metrics
"State memory keeps growing in production."	Missing or ineffective watermark/timeout; check the grouping includes the event-time window; switch to RocksDB; store aggregates not raw events
"Business wants real time but reads the report once a day."	Propose incremental batch (availableNow every 15–60 min) — same freshness for their use, lower cost and complexity
"One Kafka partition is far behind the others."	Hot key or skew — check key distribution, change key or add salt, increase partitions, or split work with **minPartitions**

Sagar

27. Streaming coding questions with answers

These are the hands-on questions asked in data engineering coding rounds and take-home tasks. Each has the question, a working answer in PySpark Structured Streaming (one in plain Python), and the points the interviewer is checking. Try each one yourself before reading the answer.

All answers assume:

```
from pyspark.sql import functions as F, Window as W
from delta.tables import DeltaTable
BROKERS = "broker1:9092,broker2:9092"
```

Q1. Read a Kafka topic, parse JSON and land it in Delta

Topic **orders.v1** has JSON values: **order_id, customer_id, amount, status, event_ts**. Land parsed records in **bronze.orders**, restartable without losing or duplicating data.

```
schema = "order_id LONG, customer_id LONG, amount DECIMAL(18,2), status STRING, event_ts
TIMESTAMP"

orders = (spark.readStream.format("kafka")
          .option("kafka.bootstrap.servers", BROKERS)
          .option("subscribe", "orders.v1")
          .option("startingOffsets", "earliest")
          .load()
          .select(F.from_json(F.col("value").cast("string"), schema).alias("o"),
                  "partition", "offset", F.col("timestamp").alias("kafka_ts"))
          .select("o.*", "partition", "offset", "kafka_ts"))

q = (orders.writeStream.format("delta")
     .option("checkpointLocation", "/chk/bronze_orders")
     .outputMode("append")
     .trigger(processingTime="1 minute")
     .toTable("bronze.orders"))
```

Checking: **value** is binary → cast to string; explicit schema; a unique checkpoint; **startingOffsets** only applies on first start; keeping partition/offset for traceability.

Q2. Events per page in 5-minute windows

From **silver.clicks (page, user_id, event_ts)**, count events per page in 5-minute tumbling windows. Events can be up to 10 minutes late. Write final results only.

```
counts = (spark.readStream.table("silver.clicks")
          .withWatermark("event_ts", "10 minutes")
          .groupBy(F.window("event_ts", "5 minutes"), "page")
          .count()
          .select(F.col("window.start").alias("window_start"),
                  F.col("window.end").alias("window_end"), "page", "count"))

(counts.writeStream.format("delta").outputMode("append")
   .option("checkpointLocation", "/chk/page_counts_5m")
   .toTable("gold.page_counts_5m"))
```

Checking: watermark **before** the aggregation on the same event-time column; append mode emits a window only after the watermark passes its end; flattening the **window** struct.

Q3. Moving average with a sliding window

Per device, compute the average temperature over the last 10 minutes, updated every minute.

```
avg_temp = (readings
  .withWatermark("reading_ts", "5 minutes")
  .groupBy("device_id", F.window("reading_ts", "10 minutes", "1 minute"))
  .agg(F.avg("temperature").alias("avg_temp_10m"), F.count("*").alias("readings")))
```

Checking: window(duration, slide) syntax; each reading belongs to 10 windows, so state and output are 10× a tumbling window — mention the cost.

Q4. Remove duplicate events

*The mobile SDK retries, so the same **event_id** can arrive several times within 30 minutes. Deduplicate without unbounded state.*

```
deduped = (events
  .withWatermark("event_ts", "30 minutes")
  .dropDuplicatesWithinWatermark(["event_id"]))          # Spark 3.5+

# Before Spark 3.5: include the event-time column so state can be cleaned
deduped_old = (events.withWatermark("event_ts", "30 minutes")
                     .dropDuplicates(["event_id", "event_ts"]))
```

Checking: plain **dropDuplicates(["event_id"])** without a watermark keeps every ID forever (memory leak). With the older API, duplicates must share the same **event_ts** to be removed.

Q5. User sessions with a 30-minute inactivity gap

Group each user's clicks into sessions that end after 30 minutes of inactivity. Output session start, end, number of events.

```
sessions = (clicks
  .withWatermark("event_ts", "10 minutes")
  .groupBy("user_id", F.session_window("event_ts", "30 minutes"))
  .agg(F.count("*").alias("events"),
       F.min("event_ts").alias("session_start"),
       F.max("event_ts").alias("last_event")))

(sessions.writeStream.format("delta").outputMode("append")
   .option("checkpointLocation", "/chk/sessions").toTable("silver.sessions"))
```

Checking: ***session_window*** *(Spark 3.2+); a session is emitted only after the watermark passes its end (last event + gap) — sessions appear with a delay. For complex session rules, use arbitrary stateful processing.*

Q6. Enrich a stream with a dimension table

Add product name and category to each click from ***silver.products*** *(Delta, updated daily).*

```
products = spark.read.table("silver.products").select("product_id", "product_name", "category")

enriched = (spark.readStream.table("silver.clicks")
            .join(F.broadcast(products), "product_id", "left"))
```

Checking: *stream–static join; the Delta table is re-read every micro-batch, so daily updates flow in; broadcast avoids a shuffle; left join keeps clicks with unknown products.*

Q7. Join impressions with clicks

Match each ad impression with a click on the same ad within 1 hour. Keep impressions with no click (for CTR).

```
imp = (impressions.select("imp_id", F.col("ad_id").alias("imp_ad_id"), "imp_ts")
                  .withWatermark("imp_ts", "2 hours"))
clk = (clicks.select("click_id", F.col("ad_id").alias("click_ad_id"), "click_ts")
             .withWatermark("click_ts", "3 hours"))

joined = imp.join(clk, F.expr("""
            click_ad_id = imp_ad_id AND
            click_ts >= imp_ts AND
            click_ts <= imp_ts + INTERVAL 1 HOUR"""), "leftOuter")
```

Checking: *watermarks on* ***both*** *streams plus a* ***time-range*** *condition — otherwise state grows forever and outer rows are never emitted. Unmatched impressions appear only after the watermark passes the 1-hour limit.*

Q8. Keep only the latest status per order (upsert)

silver.order_events *streams status changes. Maintain* ***gold.orders_current*** *with one row per order and its latest status. Events can arrive out of order.*

```
def upsert(batch_df, batch_id):
    latest = (batch_df
      .withColumn("rn", F.row_number().over(
          W.partitionBy("order_id").orderBy(F.desc("event_ts"))))
      .filter("rn = 1").drop("rn"))
    (DeltaTable.forName(batch_df.sparkSession, "gold.orders_current").alias("t")
       .merge(latest.alias("s"), "t.order_id = s.order_id")
       .whenMatchedUpdateAll(condition="s.event_ts > t.event_ts")
       .whenNotMatchedInsertAll()
       .execute())

(spark.readStream.table("silver.order_events")
   .writeStream.foreachBatch(upsert)
   .option("checkpointLocation", "/chk/orders_current")
   .trigger(processingTime="1 minute").start())
```

Checking: window functions aren't allowed on a streaming DataFrame directly, but inside **foreachBatch** the batch is a normal DataFrame; dedup within the batch before MERGE; the timestamp condition stops older events overwriting newer state; idempotent on retries.

Q9. Apply CDC events from a database

A topic carries Debezium-style events **{op, before, after, lsn}** for **customers**. Keep **silver.customers** in sync, including deletes.

```
cdc_schema = """op STRING, lsn STRING,
                before STRUCT<id: LONG, name: STRING, city: STRING>,
                after  STRUCT<id: LONG, name: STRING, city: STRING>"""

def apply_cdc(batch_df, batch_id):
    ev = (batch_df.select(F.from_json(F.col("value").cast("string"),
cdc_schema).alias("e")).select("e.*")
          .withColumn("id", F.coalesce("after.id", "before.id")))
    latest = (ev.withColumn("rn",
F.row_number().over(W.partitionBy("id").orderBy(F.desc("lsn"))))
                .filter("rn = 1")
                .select("id", "op", "lsn", "after.name", "after.city"))
    (DeltaTable.forName(batch_df.sparkSession, "silver.customers").alias("t")
       .merge(latest.alias("s"), "t.id = s.id")
       .whenMatchedDelete(condition="s.op = 'd' AND s.lsn > t.lsn")
       .whenMatchedUpdate(condition="s.op <> 'd' AND s.lsn > t.lsn",
                          set={"name": "s.name", "city": "s.city", "lsn": "s.lsn"})
       .whenNotMatchedInsert(condition="s.op <> 'd'",
                             values={"id": "s.id", "name": "s.name", "city": "s.city", "lsn": "s.lsn"})
       .execute())

(spark.readStream.format("kafka").option("kafka.bootstrap.servers", BROKERS)
   .option("subscribe", "crm.customers").load()
   .writeStream.foreachBatch(apply_cdc)
   .option("checkpointLocation", "/chk/customers_cdc").start())
```

Checking: order by source position (LSN), not arrival; latest per key per batch; only apply newer LSNs (replays are harmless); deletes take the key from **before**. Note: LSNs must be in a sortable, fixed-width format.

Q10. Top 3 pages per 5-minute window

Find the top 3 pages by views in each 5-minute window, continuously.

```
# Sorting/limit/ranking aren't supported on streaming aggregates in append mode.
# Aggregate in the stream, rank inside foreachBatch.
views = (clicks.withWatermark("event_ts", "10 minutes")
               .groupBy(F.window("event_ts", "5 minutes"), "page").count())

def top3(batch_df, batch_id):
    w = W.partitionBy("window").orderBy(F.desc("count"))
    (batch_df.withColumn("rnk", F.row_number().over(w)).filter("rnk <= 3")
       .write.format("delta").mode("append").saveAsTable("gold.top_pages_5m"))

(views.writeStream.outputMode("append").foreachBatch(top3)
   .option("checkpointLocation", "/chk/top_pages").start())
```

Checking: knowing what streaming doesn't support (**orderBy**, **limit**, window functions on streams) and the **foreachBatch** workaround. With append mode, each window reaches **foreachBatch** once, when final.

Q11. Distinct users per window

Count distinct users per 1-minute window.

```
# countDistinct is not supported on streaming DataFrames
uniq = (clicks.withWatermark("event_ts", "5 minutes")
              .groupBy(F.window("event_ts", "1 minute"))
              .agg(F.approx_count_distinct("user_id").alias("approx_users")))

# exact alternative (Spark 3.4+): one row per (window, user), then count rows per window
per_user = (clicks.withWatermark("event_ts", "5 minutes")
                  .groupBy(F.window("event_ts", "1 minute").alias("w"), "user_id").count())
exact = (per_user.groupBy(F.window(F.window_time("w"), "1 minute").alias("w"))
                 .agg(F.count("*").alias("users")))
```

Checking: **countDistinct** isn't supported in streaming; approximate (HyperLogLog) is usual; the exact version chains two time-window aggregations using **window_time** (Spark 3.4+) — correct, but keeps much more state. (A plain **dropDuplicates(["w", "user_id"])** would never clean its state, because the watermark column isn't in the keys.)

Q12. Alert when a device is too hot

Raise an alert when a device's temperature is above 80 °C in at least 3 readings within 10 minutes. Don't alert the same device more than once per 30 minutes.

```
hot = (readings.filter("temperature > 80")
       .withWatermark("reading_ts", "5 minutes")
       .groupBy("device_id", F.window("reading_ts", "10 minutes"))
       .agg(F.count("*").alias("hot_readings"))
       .filter("hot_readings >= 3"))

def raise_alerts(batch_df, batch_id):
    recent = (spark.table("gold.device_alerts")
              .filter("alert_ts > current_timestamp() - INTERVAL 30 MINUTES")
              .select("device_id"))
    new = (batch_df.select("device_id", "window", "hot_readings")
                   .dropDuplicates(["device_id"])
                   .join(recent, "device_id", "left_anti")
                   .withColumn("alert_ts", F.current_timestamp())
                   .withColumn("batch_id", F.lit(batch_id)))
    new.write.format("delta").mode("append").saveAsTable("gold.device_alerts")

(hot.writeStream.outputMode("append").foreachBatch(raise_alerts)
    .option("checkpointLocation", "/chk/device_alerts").start())
```

Checking: filtering before aggregating (less state); windowed count; cool-down using a **left_anti** join against recent alerts; storing **batch_id** to detect retries.

Q13. Send bad records to a quarantine table

Some Kafka messages aren't valid JSON or miss required fields. Keep good records flowing and park bad ones.

```
parsed = (raw.select(F.col("value").cast("string").alias("raw"), "partition", "offset")
             .withColumn("o", F.from_json("raw", schema)))

good = (parsed.filter("o IS NOT NULL AND o.order_id IS NOT NULL AND o.event_ts IS NOT
NULL")
              .select("o.*", "partition", "offset"))
bad  = (parsed.filter("o IS NULL OR o.order_id IS NULL OR o.event_ts IS NULL")
              .select("raw", "partition", "offset", F.current_timestamp().alias("quarantined_at")))

good.writeStream.format("delta").option("checkpointLocation", "/chk/good").toTable("silver.orders")
bad.writeStream.format("delta").option("checkpointLocation",
"/chk/bad").toTable("quarantine.orders")
```

Checking: **from_json** returns null (or a struct of nulls) for bad input — it doesn't fail; never block the stream on bad data; separate checkpoints per query; keep the raw value for replay after a fix.

Q14. Run a stream as a scheduled incremental job

New JSON files land in a folder all day. Process them every hour, then stop the job to save cost.

```
(spark.readStream.schema(schema)
      .option("maxFilesPerTrigger", 1000)
      .json("abfss://raw@lake.dfs.core.windows.net/orders/")
 .writeStream.format("delta")
      .option("checkpointLocation", "/chk/orders_hourly")
      .trigger(availableNow=True)
      .toTable("bronze.orders")
 .awaitTermination())
```

Checking: **availableNow** processes everything new, possibly in several batches, then stops; the checkpoint remembers which files were processed; scheduled by Airflow/ADF/Databricks Jobs. On Databricks, Auto Loader (**format("cloudFiles")**) scales file discovery better.

Q15. Stream changes (including updates and deletes) from a Delta table

silver.orders gets MERGE updates. Stream every change to a downstream table.

```
spark.sql("ALTER TABLE silver.orders SET TBLPROPERTIES (delta.enableChangeDataFeed = true)")

changes = (spark.readStream.format("delta")
           .option("readChangeFeed", "true")
           .option("startingVersion", 120)
           .table("silver.orders")
           .filter("_change_type IN ('insert', 'update_postimage', 'delete')"))
```

Checking: a plain Delta stream only handles appends and fails on updates/deletes; change data feed gives **_change_type**, **_commit_version**, **_commit_timestamp**; skip **update_preimage** unless you need old values.

Q16. Running total per customer that survives retries

Maintain total spend per customer from a payments stream. Retries must not double-count.

```
def add_totals(batch_df, batch_id):
    sums = batch_df.groupBy("customer_id").agg(F.sum("amount").alias("amt"))
    tgt = DeltaTable.forName(batch_df.sparkSession, "gold.customer_spend")
    (tgt.alias("t").merge(sums.alias("s"), "t.customer_id = s.customer_id")
        .whenMatchedUpdate(condition=f"t.last_batch_id < {batch_id}",
                           set={"total": "t.total + s.amt", "last_batch_id": F.lit(batch_id)})
        .whenNotMatchedInsert(values={"customer_id": "s.customer_id", "total": "s.amt",
                                      "last_batch_id": F.lit(batch_id)})
        .execute())

payments.writeStream.foreachBatch(add_totals).option("checkpointLocation", "/chk/spend").start()
```

Checking: "add to total" is **not idempotent** — a retried batch would add twice. Recording **last_batch_id** per row (or using Delta's **txnAppId/txnVersion** options on writes) makes it safe. Alternative: use a streaming aggregation (**groupBy().sum()** in update mode) and let Spark's state handle it.

Q17. Write a stream to Kafka

*Publish enriched orders to **orders.enriched.v1**, keyed by **order_id**.*

```
(enriched.select(F.col("order_id").cast("string").alias("key"),
                 F.to_json(F.struct("*")).alias("value"))
   .writeStream.format("kafka")
   .option("kafka.bootstrap.servers", BROKERS)
   .option("topic", "orders.enriched.v1")
   .option("checkpointLocation", "/chk/orders_enriched")
   .start())
```

Checking: *the Kafka sink needs **key/value** columns (string or binary); key keeps per-order ordering; Kafka sink is at-least-once — downstream consumers dedup by key/event ID.*

Q18. Start from a point in time and limit batch size

After an incident, reprocess a topic from 2026-01-15 00:00 UTC with a new checkpoint, without overloading the cluster.

```
replay = (spark.readStream.format("kafka")
          .option("kafka.bootstrap.servers", BROKERS)
          .option("subscribe", "orders.v1")
          .option("startingTimestamp", "1768435200000")      # epoch millis for 2026-01-
15T00:00:00Z
          .option("maxOffsetsPerTrigger", 200_000)
          .load())
```

Checking: **startingTimestamp** *(or **startingOffsetsByTimestamp** per partition) applies only to a new checkpoint; rate limiting keeps batches stable during catch-up; the sink must be idempotent because of the overlap with already-processed data.*

Q19. Monitor and stop a stream safely

Print progress every minute and stop the stream gracefully when a stop-marker file appears.

```
import time, json
q = df.writeStream.format("delta").option("checkpointLocation", "/chk/q").toTable("t")

while q.isActive:
    p = q.lastProgress
    if p:
        print(json.dumps({"batch": p["batchId"], "in_rps": p["inputRowsPerSecond"],
                          "proc_rps": p["processedRowsPerSecond"],
                          "ms": p["durationMs"].get("triggerExecution")}))
    if any(f.name == "stop_q" for f in dbutils.fs.ls("/ctl/")):      # marker file present
        while q.status["isTriggerActive"]:
            time.sleep(1)
        q.stop()
    time.sleep(60)
```

Checking: ***lastProgress*** *metrics (input vs processing rate, duration); waiting for* ***isTriggerActive*** *to be false before* ***stop()****; checkpoint makes restart safe either way. (Off Databricks, check the marker with your storage SDK or* ***os.path.exists****.)*

Q20. Kafka consumer in plain Python with safe commits

Without Spark: consume ***orders.v1****, insert into PostgreSQL in batches, never lose data, and tolerate duplicates.*

```
from confluent_kafka import Consumer
import json, psycopg

c = Consumer({"bootstrap.servers": BROKERS, "group.id": "orders-to-pg",
              "enable.auto.commit": False, "auto.offset.reset": "earliest"})
c.subscribe(["orders.v1"])

with psycopg.connect(PG_DSN) as conn:
    while True:
        msgs = c.consume(num_messages=500, timeout=1.0)
        rows = [json.loads(m.value()) for m in msgs if m.error() is None]
        if not rows:
            continue
        with conn.cursor() as cur:
            cur.executemany("""INSERT INTO orders (order_id, status, amount, event_ts)
                               VALUES (%(order_id)s, %(status)s, %(amount)s, %(event_ts)s)
                               ON CONFLICT (order_id) DO UPDATE
                               SET status = EXCLUDED.status, amount = EXCLUDED.amount,
                                   event_ts = EXCLUDED.event_ts
                               WHERE orders.event_ts < EXCLUDED.event_ts""", rows)
        conn.commit()                          # 1. data committed
        c.commit(asynchronous=False)           # 2. then offsets → at-least-once, upsert makes it
idempotent
```

Checking: *manual commit* ***after*** *the database commit (at-least-once); upsert with an event-time guard makes duplicates and out-of-order events harmless; batching for throughput. A crash between steps 1 and 2 replays the batch safely.*

Q21. Test a streaming transformation

Write a test proving duplicates are removed and late events beyond the watermark are dropped.

```
def test_dedup_stream(spark, tmp_path):
    src, chk, out = str(tmp_path / "in"), str(tmp_path / "chk"), str(tmp_path / "out")
    spark.createDataFrame([("e1", "2026-01-15T10:00:00"), ("e1", "2026-01-15T10:00:00"),
                           ("e2", "2026-01-15T10:01:00")], "event_id string, event_ts string") \
         .write.json(src)

    q = (spark.readStream.schema("event_id string, event_ts timestamp").json(src)
           .withWatermark("event_ts", "10 minutes")
           .dropDuplicatesWithinWatermark(["event_id"])
           .writeStream.format("parquet").option("checkpointLocation", chk)
           .trigger(availableNow=True).start(out))
    q.awaitTermination()
    assert spark.read.parquet(out).count() == 2
```

Checking: *using a file source + **availableNow** to run a stream to completion in tests; temp directories for input, checkpoint and output; asserting on the output. Extend it by writing a second file and rerunning with the same checkpoint to test restarts and late data.*

What isn't supported on streaming DataFrames

Knowing these saves you in live coding rounds:

Not supported (directly)	What to do instead
orderBy / **sort** (except on aggregates in complete mode)	Sort inside **foreachBatch**, or in the sink/query layer
limit / **take(n)**	**foreachBatch**, or limit when reading the output
countDistinct	**approx_count_distinct**, or dedup then count
Window functions (**row_number**, **lag**) over a stream	**foreachBatch** (batch DataFrame), or stateful processing
distinct() without bounds	**dropDuplicates** with a watermark
Actions like **count()**, **collect()**, **show()**	Write to a sink; use **foreachBatch** or the memory sink for debugging
Outer stream–stream joins without watermarks	Add watermarks on both sides + time-range condition
complete mode without aggregation	Use append/update

Memory hook: *"If it needs to see the whole table, a stream can't do it directly — do it per batch (**foreachBatch**) or per window."*

28. Glossary

sagar

Term	Meaning
Unbounded data	Data that never ends — a stream
Event	An immutable record of something that happened, with a timestamp and key
Event time	When the event happened
Processing time	When the system processed the event
Window	A bucket of events by time (tumbling, sliding, session)
Watermark	Estimate of event-time progress; max event time minus allowed lateness
Late event	Event arriving after the watermark passed its time
State	Information remembered across events (counts, last values, buffered rows)
State store	Storage for state (memory, RocksDB), checkpointed for recovery
Checkpoint	Durable record of progress (offsets, commits, state)
Micro-batch	Processing a stream as a series of small batches
Trigger	When the next micro-batch runs
Output mode	What is written each batch (append, update, complete)
Offset	Position in a partitioned log
Consumer lag	How far a consumer is behind the latest data
At-least-once	No loss, possible duplicates
Exactly-once / effectively-once	Each event affects the result once
Idempotent	Applying the same write twice gives the same result
Backpressure / rate limiting	Controlling intake so processing isn't overwhelmed
DLQ	Dead-letter queue/topic for records that can't be processed
CDC	Change Data Capture — stream of inserts, updates, deletes from a database log
LSN / binlog position	Database log position used to order CDC events
Changelog	A stream of changes to a table
Compaction	Keeping only the latest record per key in a log
Savepoint	Manual snapshot of a Flink job's state for upgrades and migrations
Session window	Window per key that closes after a gap of inactivity
Stream–static join	Joining a stream with a table read each batch
Stream–stream join	Joining two streams with buffered state
Lambda / Kappa	Two-path (batch + speed) vs single-path streaming architectures

Streaming in one sentence: **process events by when they happened, wait a measured amount for late ones, remember only what you need and forget on time, record your progress, and make every write safe to repeat.**

— Sagar

Sagar